Clarifying rules for brace elision in aggregate initialization

Document #:	P3106R1
Date:	2024-03-21
Project:	Programming Language C++
Audience:	CWG

Reply-to:	James Touton <bekenn@gmail.com>

Contents

§ 1 Introduction

[CWG2149] points out an inconsistency in the wording with respect to array lengths inferred from braced initializer lists in the presence of brace elision. The wording has changed since the issue was raised, but the essence of the issue remains. The standard first states that the length of an array of unknown bound is the same as the number of elements in the initializer list, but then describes the rules for brace elision, in which some elements are used to initialize subobjects of aggregate members, such that there is not a one-to-one mapping of initializer list elements to array elements. Similarly, aggregate initialization from a braced initializer list is supposed to explicitly initialize a number of aggregate elements equal to the length of the list, which is not possible in the presence of brace elision.

This paper aims to resolve the core issue by clarifying the rules for brace elision to match user expectations and the behavior of existing compilers. The intent is to better describe the existing design without introducing any evolutionary changes.

§ 2 Wording

All changes are presented relative to [N4971].

§9.4.2 [dcl.init.aggr]:

Modify the second list item of paragraph 3:

(3.2) — If the initializer list is a brace-enclosed *initializer-list*, the explicitly initialized elements of the aggregate are ~~the first *n* elements of the aggregate, where *n* is the number of elements in the initializer list~~ those for which an element of the initializer list appertains to the aggregate element or to a subobject thereof (see below).

Modify paragraph 4:

4 For each explicitly initialized element:

(4.1) — *[...]*

(4.2) — Otherwise, if the initializer list is a brace-enclosed *designated-initializer-list*, the element is ~~copy-initialized from the corresponding *initializer-clause* or is~~ initialized with the *brace-or-equal-initializer* of the corresponding *designated-initializer-clause*. If that initializer is of the form ~~*assignment-expression* or~~ `=` *assignment-expression* and a narrowing conversion (9.4.5 [dcl.init.list]) is required to convert the expression, the program is ill-formed.

[*Note:* ~~If the initialization is by *designated-initializer-clause*, its~~ The form of the initializer determines whether copy-initialization or direct-initialization is performed. — *end note*]

(4.3) — Otherwise, the initializer list is a brace-enclosed *initializer-list*. If an *initializer-clause* appertains to the aggregate element, then the aggregate element is copy-initialized from the *initializer-clause*. Otherwise, the aggregate element is copy-initialized from a brace-enclosed *initializer-list* consisting of all of the *initializer-clauses* that appertain to subobjects of the aggregate element, in the order of appearance.

[*Note:* If an initializer is itself an initializer list, *[...]* — *end note*]

[*Example: [...]* — *end example*]

Modify paragraph 6:

6 [*Example:*

```
struct S { int a; const char* b; int c; int d = b[a]; };
S ss = { 1, "asdf" };
```

initializes `ss.a` with 1, `ss.b` with `"asdf"`, `ss.c` with the value of an expression of the form `int{}` (that is, `0`), and `ss.d` with the value of `ss.b[ss.a]` (that is, `'s'`), and in

~~struct X { int i, j, k = 42; };~~
~~X a[] = { 1, 2, 3, 4, 5, 6 };~~
~~X b[2] = { { 1, 2, 3 }, { 4, 5, 6 } };~~

~~a and b have the same value~~

```
struct A {
  string a;
  int b = 42;
  int c = -1;
};
```

`A{.c=21}` has the following steps:

(6.1) — Initialize `a` with `{}`

(6.2) — Initialize `b` with `= 42`

(6.3) — Initialize `c` with `= 21`

— *end example*]

Modify paragraph 10:

10 ~~An~~ The number of elements (9.3.4.5 [dcl.array]) in an array of unknown bound initialized with a brace-enclosed *initializer-list* ~~containing *n initializer-clauses*~~ is ~~defined as having `n` elements (9.3.4.5 [dcl.array])~~ the number of explicitly initialized elements of the array.

[*Example:*

```
int x[] = { 1, 3, 5 };
```

declares and initializes `x` as a one-dimensional array that has three elements since no size was specified and there are three initializers. — *end example*]

[*Example:* In

```
struct X { int i, j, k; };
X a[] = { 1, 2, 3, 4, 5, 6 };
X b[2] = { { 1, 2, 3 }, { 4, 5, 6 } };
```

`a` and `b` have the same value. — *end example*]

An array of unknown bound shall not be initialized with an empty *braced-init-list* `{}`. [*Footnote:* The syntax provides for empty *braced-init-list*s, but nonetheless C++ does not have zero length arrays. — *end footnote*]

[*Note:* A default member initializer does not determine the bound for a member array of unknown bound. Since the default member initializer is ignored if a suitable *mem-initializer* is present (11.9.3 [class.base.init]), the

default member initializer is not considered to initialize the array of unknown bound.

[*Example:*

```
struct S {
   int y[] = { 0 };             // error: non-static data
         member of incomplete type
};
```

— *end example*]
— *end note*]

Remove paragraph 12:

12 ~~An *initializer-list* is ill-formed if the number of *initializer-clause*s exceeds the number of elements of the aggregate.~~

~~[*Example:*~~

```
char cv[4] = { 'a', 's', 'd', 'f', 0 };      // error
```

~~is ill-formed. — *end example*]~~

Remove paragraph 14:

14 ~~If an aggregate class `C` contains a subaggregate element `e` with no elements, the *initializer-clause* for `e` shall not be omitted from an *initializer-list* for an object of type `C` unless the *initializer-clause*s for all elements of `C` following `e` are also omitted.~~

~~[*Example: […] — end example*]~~

The example in the above paragraph is relocated to another paragraph below.

Remove paragraph 16:

16 ~~Braces can be elided in an *initializer-list* as follows. If the *initializer-list* begins with a left brace, then the succeeding comma-separated list of *initializer-clause*s initializes the elements of a subaggregate; it is erroneous for there to be more *initializer-clause*s than elements. If, however, the *initializer-list* for a subaggregate does not begin with a left brace, then only enough *initializer-clause*s from the list are taken to initialize the elements of the subaggregate; any remaining *initializer-clause*s are left to initialize the next element of the aggregate of which the current subaggregate is an element.~~

~~[*Example: [...] — end example*]~~

The example in the above paragraph is relocated to another paragraph below.

Insert a new paragraph in place of removed paragraph 16:

14 Each *initializer-clause* in a brace-enclosed *initializer-list* is said to *appertain* to an element of the aggregate being initialized or to an element of one of its subaggregates. Considering the sequence of *initializer-clause*s, and the sequence of aggregate elements initially formed as the sequence of elements of the aggregate being initialized and potentially modified as described below, each *initializer-clause* appertains to the corresponding aggregate element if

(14.1) — the aggregate element is not an aggregate, or

(14.2) — the *initializer-clause* begins with a left brace, or

(14.3) — the *initializer-clause* is an expression and an implicit conversion sequence can be formed that converts the expression to the type of the aggregate element, or

(14.4) — the aggregate element is an aggregate that itself has no aggregate elements.

Otherwise, the aggregate element is an aggregate and that subaggregate is replaced in the list of aggregate elements by the sequence of its own aggregate elements, and the appertainment analysis resumes with the first such element and the same *initializer-clause*.

[*Note:* These rules apply recursively to the aggregate's subaggregates.

[*Example:* In

```
struct S1 { int a, b; };
struct S2 { S1 s, t; };

S2 x[2] = { 1, 2, 3, 4, 5, 6, 7, 8 };
S2 y[2] = {
  {
    { 1, 2 },
    { 3, 4 }
  },
  {
    { 5, 6 },
    { 7, 8 }
  }
};
```

x and y have the same value. — *end example*]

— end note]

This process continues until all *initializer-clause*s have been exhausted.

If any *initializer-clause* remains that does not appertain to an element of the aggregate or one of its subaggregates, the program is ill-formed.

[*Example:*

```
char cv[4] = { 'a', 's', 'd', 'f', 0 };      // error: too
        many initializers
```

— end example]

Remove paragraph 17:

15 ~~All implicit type conversions (7.3 [conv]) are considered when initializing the element with an *assignment-expression*. If the *assignment-expression* can initialize an element, the element is initialized. Otherwise, if the element is itself a subaggregate, brace elision is assumed and the *assignment-expression* is considered for the initialization of the first element of the subaggregate.~~

~~[*Note:* As specified above, brace elision cannot apply to subaggregates with no elements; an *initializer-clause* for the entire subobject is required. *— end note*]~~

~~[*Example:*~~

```
struct A {
  int i;
  operator int();
};
struct B {
  A a1, a2;
  int z;
};
A a;
B b = { 4, a, a };
```

~~Braces are elided around the *initializer-clause* for `b.a1.i`. `b.a1.i` is initialized with 4, `b.a2` is initialized with `a`, `b.z` is initialized with whatever `a.operator int()` returns. *— end example*]~~

Insert the remaining new paragraphs (containing the examples relocated from prior paragraphs) after the above paragraph:

16 [*Example:*

```
float y[4][3] = {
  { 1, 3, 5 },
  { 2, 4, 6 },
  { 3, 5, 7 },
};
```

is a completely-braced initialization: 1, 3, and 5 initialize the first row of the array `y[0]`, namely `y[0][0]`, `y[0][1]`, and `y[0][2]`. Likewise the next two lines initialize `y[1]` and `y[2]`. The initializer ends early and therefore `y[3]`'s elements are initialized as if explicitly initialized with an expression of the form `float()`, that is, are initialized with `0.0`. In the following example, braces in the *initializer-list* are elided; however the *initializer-list* has the same effect as the completely-braced *initializer-list* of the above example,

```
float y[4][3] = {
  1, 3, 5, 2, 4, 6, 3, 5, 7
};
```

The initializer for `y` begins with a left brace, but the one for `y[0]` does not, therefore three elements from the list are used. Likewise the next three are taken successively for `y[1]` and `y[2]`. — *end example*]

17 [*Note:* The initializer for an empty subaggregate is required if any initializers are provided for subsequent elements.

[*Example:*

```
struct S { } s;
struct A {
  S s1;
  int i1;
  S s2;
  int i2;
  S s3;
  int i3;
} a = {
  { },              // Required initialization
  0,
  s,                // Required initialization
  0
};                  // Initialization not required for
        A::s3 because A::i3 is also not initialized
```

— *end example*]
— *end note*]

18 [*Example:*

```
struct A {
  int i;
  operator int();
};
struct B {
  A a1, a2;
  int z;
};
A a;
B b = { 4, a, a };
```

Braces are elided around the *initializer-clause* for `b.a1.i`. `b.a1.i` is initialized with 4, `b.a2` is initialized with `a`, `b.z` is initialized with whatever `a.operator int()` returns. — *end example*]

§ 3 References

[CWG2149] Vinny Romano. 2015-06-25. Brace elision and array length deduction.
https://wg21.link/cwg2149

[N4971] Thomas Köppe. 2023-12-18. Working Draft, Programming Languages — C++.
https://wg21.link/n4971